

1 Tối ưu trên pixel thay vì trên trọng số (Optimizing Pixels, Not Weights)

PHỤ THUỘC C/12-co-che-huan-luyen/01-co-hoc-cua-gradient (gradient chảy về đâu là do ta chọn); C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh (hàm mất mát quyết định kiểu thất bại); C/13-kien-truc-backbone/01-dong-cnn (bản đồ đặc trưng theo tầng).

TÓM TẮT

Có một ý tưởng đơn giản nối liền năm kỹ thuật trông chẳng liên quan gì tới nhau: đóng băng một mạng đã huấn luyện, coi *ảnh* là biến cần tối ưu, rồi hạ gradient trên chính pixel. Đối hàm mục tiêu là được một kỹ thuật khác — đảo ngược đặc trưng, DeepDream, trực quan hoá lớp, mẫu đối kháng, và chuyển phong cách. Đọc xong bạn viết được cả năm bằng cùng một vòng lặp mười dòng, và hiểu vì sao chuyển phong cách phiên bản tiến thẳng là ví dụ sạch nhất của mẫu hình “đẩy chi phí từ lúc chạy sang lúc huấn luyện”. Nếu chỉ nhớ một điều: tối ưu trên pixel tìm ra đúng những hướng mà mạng và mắt người *bất đồng*; khi hai bên tình cờ đồng ý ta gọi kết quả là nghệ thuật, khi bất đồng ta gọi nó là mẫu đối kháng — cùng một cơ chế.

KIẾN THỨC CẦN CÓ TRƯỚC

- **lan truyền ngược (backpropagation)** — thuật toán tính đạo hàm của một đại lượng đầu ra theo mọi nút của một đồ thị tính toán khả vi. Nó không phân biệt “tham số” với “dữ liệu” — đó là tiền đề của cả mục này.
- **bản đồ đặc trưng (feature map)** — tensor đầu ra của một tầng tích chập: một chồng “ảnh” nhỏ hơn, mỗi kênh phản ứng với một loại mẫu hình thị giác.
- **đóng băng (freeze)** — đặt cờ không tính gradient cho trọng số của một mạng, để nó chỉ còn đóng vai một hàm chấm điểm cố định.
- **ma trận Gram** — ma trận tích trong giữa các kênh của một bản đồ đặc trưng, cộng dồn trên mọi vị trí không gian; nó mô tả “kênh nào hay bật cùng kênh nào” mà bỏ hẳn thông tin vị trí.
- **mẫu đối kháng (adversarial example)** — ảnh bị thêm một nhiễu rất nhỏ, mắt người không thấy, nhưng đủ để mạng phân loại sai hoàn toàn.
- **hàm mục tiêu (objective function)** — toàn bộ đại lượng được tối ưu, gồm phần đo sai lệch (**hàm mất mát (loss function)**) cộng các số hạng chính quy hoá. Mục này dùng đúng từ “hàm mục tiêu” vì phần chính quy hoá ở đây quyết định kết quả không kém phần đo sai lệch.
- **chính quy hoá (regularization)** — phần cộng thêm vào hàm mục tiêu để loại bỏ các nghiệm “đúng về số nhưng vô lý về mặt ảnh”, ví dụ phạt biến thiên toàn phần để ép ảnh mượt.
- **feed-forward (tiến thẳng)** — chạy dữ liệu qua mạng đúng một lần từ đầu tới cuối, không lặp — đối lập với một vòng tối ưu nhiều trăm bước.

A. Đặt vấn đề

Trong huấn luyện thông thường, dữ liệu là hằng số và trọng số là ẩn số: ta cố định ảnh, tính gradient của hàm mất mát theo trọng số, rồi cập nhật trọng số. Nhưng **lan truyền ngược (backpropagation)** không quan tâm ta gọi cái gì là ẩn số — nó tính đạo hàm dọc theo một đồ thị khả vi, và đồ thị đó có đầu vào cũng là một nút. Vậy hoàn toàn có thể làm ngược lại: cố định trọng số, tính gradient theo *pixel đầu vào*, rồi cập nhật ảnh.

Câu hỏi thú vị không phải “làm được không” mà “để làm gì”. Câu trả lời: một mạng đã huấn luyện là một hàm chấm điểm ngầm định của thế giới thị giác — nó đã học được cái gì là mèo, cái gì là hoa văn, cái gì là biên. Tối ưu trên pixel là cách ép hàm chấm điểm đó *nói ra* thứ nó biết, dưới dạng một bức ảnh. Hình 1 là toàn bộ sự khác biệt, vẽ trong một hình.

Học mục này thì làm được gì mà trước đó không làm được? Bốn việc: cài chuyển phong cách, DeepDream, trực quan hoá lớp và tấn công đối kháng bằng cùng một đoạn code; giải thích được

vì sao ma trận Gram bắt được “phong cách”; nhận ra mẫu hình đời chi phí sang lúc huấn luyện khi gặp lại nó ở **chưng cất tri thức** (*knowledge distillation*); và có một công cụ chẩn đoán mạnh — muốn biết mạng của bạn thật sự nhìn cái gì, hãy tối ưu một ảnh làm nó phát diện rồi nhìn ảnh đó.



Hình 1. Cùng một đồ thị khả vi, hai cách dùng. Lan truyền ngược không phân biệt “dữ liệu” với “tham số” — nó chỉ tính đạo hàm dọc đồ thị; việc chọn nút nào là biến tối ưu hoàn toàn nằm ở người viết. Đây là lý do nằm kỹ thuật ở Bảng 1 dùng chung một đoạn code.

→ **trường hợp riêng của** C/12/01 — tối ưu trên pixel chỉ là lan truyền ngược chạy tới tận nút đầu vào thay vì dừng ở nút tham số.

TRỰC GIÁC

Hãy hình dung mạng đã huấn luyện là một giám khảo hội họa rất nhất quán nhưng không biết nói — ông ta chỉ chấm điểm, không giải thích. Tối ưu trên pixel là điêu khắc một bức tượng bằng cách liên tục hỏi “sửa chỗ nào thì điểm tăng” rồi sửa đúng chỗ đó. Bức tượng cuối cùng phản ánh chính xác thứ mà giám khảo quan tâm — kể cả những thứ vô lý mà ông ta quan tâm nhưng con người thì không. Khi những thứ đó trùng với thẩm mỹ của ta, ta gọi kết quả là chuyển phong cách; khi chúng lệch hẳn, ta gọi kết quả là mẫu đối kháng.

B. Một vòng lặp, năm kỹ thuật

Vòng lặp chung ngắn tới mức viết ra được trọn vẹn:

```
# Dong bang mang; bien toi uu la anh, khong phai net.parameters()
for p in net.parameters():
    p.requires_grad_(False)
x = init_image.clone().requires_grad_(True)
opt = torch.optim.LBFGS([x])
def step():
    opt.zero_grad()
    loss = objective(net, x) # doi objective -> doi ky thuat
    loss.backward() # gradient chay ve tan x
    return loss
for _ in range(300):
    opt.step(step)
```

Chỉ dòng `objective` thay đổi giữa năm kỹ thuật ở Bảng 1. Đây không phải một sự trùng hợp thú vị mà là một bài học về cách đọc tài liệu: khi năm phương pháp chia chung một khung, học khung rẻ hơn học năm phương pháp gấp nhiều lần.

Bảng 1. Năm kỹ thuật cùng một vòng lặp, khác nhau đúng ở hàm mục tiêu.

Kỹ thuật	Hàm mục tiêu	Nó tiết lộ điều gì / hỏng khi nào
Đảo ngược đặc trưng	Khớp bản đồ đặc trưng của một ảnh mục tiêu ở một tầng	Tầng càng sâu càng mất thông tin vị trí; ảnh khôi phục ngày càng “đúng nội dung, sai bố cục”
DeepDream	Cực đại hoá chuẩn của activation ở một tầng	Cho thấy tầng đó thích hoa văn gì; kết quả lặp lại một mô-típ tới mức bệnh hoạn
Trực quan hoá lớp	Cực đại hoá điểm số của một lớp + phạt chính quy hoá	Không có phạt thì ra nhiều có điểm số cao — bằng chứng trực tiếp rằng mạng không nhìn như người
Mẫu đối kháng	Giảm điểm số lớp đúng với ràng buộc $\ \delta\ $ nhỏ	Tồn tại nhiều vô hình đối hản dự đoán; đây là chế độ hỏng của <i>mọi</i> mạng, không phải của một mạng kém
Chuyển phong cách	Nội dung ở tầng sâu + thống kê phong cách ở nhiều tầng nông	Phong cách nắm được là <i>kết cấu</i> , không phải bố cục hay ngữ nghĩa

C. Neural Style Transfer: vì sao ma trận Gram lại là “phong cách”

Bài toán của Gatys: giữ *nội dung* của ảnh A và *phong cách* của ảnh B [1]. Nội dung dễ định nghĩa — bản đồ đặc trưng ở một tầng sâu, vì tầng sâu giữ “có gì ở đâu” mà bỏ bớt chi tiết bề mặt. Phong cách mới là chỗ khó: nó phải là thứ mô tả “chất” của bức tranh mà không mô tả nội dung, tức là phải *độc lập với vị trí*.

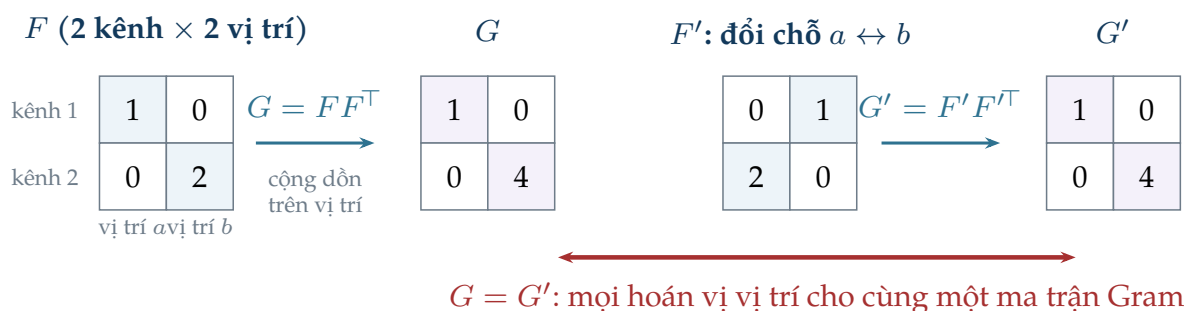
Lời giải là **ma trận Gram** (*Gram matrix*). Gọi $F \in \mathbb{R}^{C \times N}$ là bản đồ đặc trưng của một tầng, với C kênh và N vị trí không gian; đặt $G = FF^\top$, tức $G_{ij} = \sum_p F_{ip}F_{jp}$. Công thức này nói gì: G_{ij} đo mức độ hai kênh i và j cùng bật lên tại cùng một vị trí, cộng dồn trên toàn ảnh — nó là thống kê *đồng xuất hiện* của các đặc trưng, và phép cộng dồn trên p đã vứt bỏ hoàn toàn thông tin vị trí.

Tính tay trên ma trận 2×2 . Hai kênh, hai vị trí:

$$F = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} \Rightarrow G = FF^\top = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}, \quad F' = \begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix} \Rightarrow G' = \begin{pmatrix} 1 & 0 \\ 0 & 4 \end{pmatrix}.$$

Đổi chỗ hai vị trí không gian mà G không đổi. Đó vừa là sức mạnh — “phong cách” đúng là thứ không phụ thuộc chỗ đặt — vừa là giới hạn: *Gram không phân biệt được bố cục*. Nó nắm được kết cấu nét cọ, bảng màu, độ thô của hạt; nó không nắm được “chân dung thì mất ở trên miệng”. Mọi thất bại quen thuộc của chuyển phong cách đều suy ra từ đây: khuôn mặt bị loang màu, bầu trời mọc nét cọ ở giữa mặt người, cấu trúc lớn của tranh gốc biến mất.

Hình 2 vẽ đúng phép tính đó.



Hình 2. Vì sao ma trận Gram bắt được “phong cách” — và vì sao nó không bao giờ bắt được bố cục. Phép cộng dồn trên các vị trí không gian biến F thành một thống kê *đồng xuất hiện giữa các kênh*, và phép cộng thì không quan tâm thứ tự. Hai bản đồ đặc trưng khác hẳn nhau về chỗ đặt cho ra cùng một G . Đó vừa là định nghĩa vừa là giới hạn của “phong cách”: nét cọ và bảng màu thì giữ được, “mắt nằm trên miệng” thì không.

Đáng dừng lại một nhịp ở chỗ này, vì đây là lần thứ hai ta gặp cùng một mẹo: G là một *tổng* trên các vị trí, tức một hàm đối xứng — đúng cấu trúc mà PointNet dùng khi lấy max trên các điểm (C/16-thi-giac-3d/02-bieu-dien-3d-va-depth). Hai lĩnh vực không liên quan, một nguyên lý: muốn bất biến với thứ tự hay vị trí thì gộp bằng một phép toán đối xứng, và cái giá luôn luôn là mất chính cái thông tin thứ tự đó.

↪ cùng nguyên lý với C/16/02 — PointNet lấy max trên tập điểm, Gram lấy tổng trên vị trí: cùng một hàm đối xứng, cùng một cái giá.

D. Dời chi phí từ lúc chạy sang lúc huấn luyện

Bản gốc của Gatys phải chạy vài trăm bước tối ưu cho *mỗi* ảnh, mất từ vài chục giây tới vài phút trên GPU — không dùng được cho video hay cho ứng dụng tương tác. Cách gỡ là một mẫu hình tổng quát, không riêng gì chuyển phong cách.

I. Mạch 2 — ba nấc của cùng một bài toán

- 1 Vấn đề** mỗi ảnh cần một vòng tối ưu riêng.

Giải pháp huấn luyện một mạng **tiền thẳng** (*feed-forward*) để làm đúng việc mà vòng tối ưu làm, dùng chính hàm mất mát của Gatys làm tín hiệu giám sát — tức là **mất mát tri giác** (*perceptual loss*) [2].

Hệ quả lúc chạy chỉ còn một lượt xuôi, nhanh hơn ba bậc độ lớn.

Giá phải trả phải huấn luyện một mạng cho *mỗi* phong cách, và mất khả năng tinh chỉnh trọng số nội dung/phong cách sau khi huấn luyện.
- 2 Vấn đề** một mạng cho một phong cách là không mở rộng được.

Giải pháp **AdaIN**: chuẩn hoá đặc trưng của ảnh nội dung về trung bình–độ lệch chuẩn 0–1 theo từng kênh, rồi *gán lại* trung bình và độ lệch chuẩn theo từng kênh của ảnh phong cách [3].

Giả định điều đáng nói nằm ở giả định ngầm: nếu chỉ cần khớp thống kê bậc nhất theo kênh đã ra phong cách, thì phần lớn nội dung của ma trận Gram (thống kê bậc hai) là dư thừa — một phát hiện thực nghiệm khá bất ngờ.

Hệ quả một mạng duy nhất làm được phong cách tùy ý, thời gian thực.

Giá phải trả chất lượng thấp hơn bản tối ưu; các phong cách có cấu trúc lớn (tranh có hình khối rõ) bị mất nhiều nhất.
- 3 Vấn đề** ta vẫn chỉ có một đánh đổi chứ chưa có bữa trưa miễn phí.

Hệ quả

mẫu hình chung: độ khó không biến mất, nó chuyển từ lúc chạy sang lúc huấn luyện — và cùng mẫu hình đó sẽ xuất hiện ở chừng cất tri thức (D/20-toi-uu-va-nen-mo-hinh) và ở việc chừng cất bộ lấy mẫu diffusion (C/17-mo-hinh-sinh/03-diffusion).

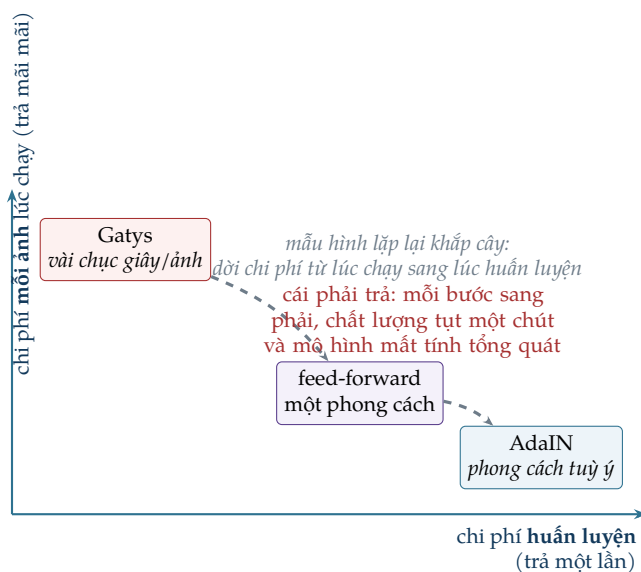
→ cùng nguyên lý với D/20 — chừng cất tri thức là chính mẫu hình này ở tầng mô hình: trả một lần lúc huấn luyện để khỏi trả mãi lúc suy luận.

→ cùng nguyên lý với C/16/03 — neural rendering làm đúng việc này trên một cảnh 3D thay vì trên một ảnh.

Bảng 2. Ba nấc của chuyển phong cách, đọc theo chỗ chi phí nằm ở đâu.

Cách làm	Chi phí mỗi ảnh	Chi phí huấn luyện	Hỏng khi nào
Gatys (tối ưu trên pixel)	Vài chục giây tới vài phút	Không có	Không dùng được cho video hay tương tác
Tiến thẳng một phong cách	Một lượt xuôi	Một lần huấn luyện cho mỗi phong cách	Số phong cách lớn; muốn đổi trọng số sau khi huấn luyện
AdaIN (phong cách tùy ý)	Một lượt xuôi	Một lần huấn luyện duy nhất	Phong cách có cấu trúc lớn; chất lượng dưới bản tối ưu

Hình 3 đặt ba nấc lên một mặt phẳng để thấy chi phí đã đi đâu.



Hình 3. Ba nấc của chuyển phong cách, vẽ trên mặt phẳng “ai trả chi phí”. Không nấc nào rẻ hơn nấc trước một cách miễn phí: cái được lấy đi ở trục dọc bị trả lại ở trục ngang, và một phần nhỏ nữa bị trả bằng chất lượng. Đây là cùng một hình sẽ đúng cho chừng cất tri thức và cho chừng cất bộ lấy mẫu diffusion — chỉ cần đổi nhãn ba hộp.

E. Giới hạn và trường hợp hỏng

Giới hạn nền tảng của cả mục này là ta đang tin vào một hàm chấm điểm mà chính ta không kiểm soát. Mạng đã huấn luyện có vô số hướng trong không gian ảnh mà nó nhạy còn mất người thì không — tối ưu trên pixel đi thẳng vào những hướng đó, vì gradient bao giờ cũng chỉ về hướng dốc nhất. Đó là lý do trực quan hoá lớp không có phạt chính quy hoá cho ra nhiều, và là lý do mẫu đối kháng tồn tại ở mọi mạng.

Hai chế độ hỏng cụ thể đáng nhớ:

- **Kết quả phụ thuộc mạnh vào khởi tạo và bộ tối ưu.** Bài toán không lỗi trên hàng trăm nghìn biến; khởi tạo từ ảnh nội dung, từ nhiễu, hay từ ảnh trắng cho ba kết quả khác nhau. Đây là tính chất của bài toán chứ không phải lỗi cài đặt — báo cáo kết quả mà không nói khởi tạo là báo cáo thiếu.
- **Chuyển phong cách mất tính nhất quán theo thời gian.** Áp cho từng khung hình video thì ảnh nhấp nháy, vì hai khung hình gần nhau rơi vào hai cực tiểu địa phương khác nhau. Cách chữa là thêm ràng buộc thời gian dựa trên dòng quang — và ta lại quay về đúng chủ đề của C/15-thi-giac-khong-thoi-gian.

CẠM BẤY

Đừng đọc một ảnh trực quan hoá lớp như bằng chứng “mạng hiểu khái niệm này”. Ảnh đó là nghiệm của một bài tối ưu có phạt chính quy hoá do bạn chọn, và phần lớn cấu trúc đẹp mắt trong đó đến từ chính phần phạt (biến thiên toàn phần, jitter, làm mượt) chứ không từ mạng. Bằng chứng: bỏ phạt đi thì ra nhiễu, dù điểm số của lớp còn cao hơn. Muốn dùng công cụ này để chẩn đoán, hãy luôn báo cáo phần phạt cùng với ảnh — xem thêm E/25-do-tin-cay-va-van-hanh về các phép kiểm tra tính đúng đắn của phương pháp giải thích [4].

CẦN NHỚ

Lan truyền ngược không phân biệt tham số với dữ liệu; chọn nút nào làm biến tối ưu là quyền của bạn. Đóng băng mạng và tối ưu trên pixel cho ra năm kỹ thuật khác nhau chỉ bằng cách đổi hàm mục tiêu — và bất cứ khi nào một vòng tối ưu lúc chạy quá đắt, hãy hỏi liệu có huấn luyện được một mạng làm thay nó hay không. Đó là mẫu hình dời chi phí sang lúc huấn luyện, gặp lại ở distillation. [T1]

F. Kết nối

Ý tưởng của mục này là bản sao chính xác của điều đã gặp ở C/16-thi-giac-3d/03-nerf-3dgs-va-hop-nhat: đặt thứ ta muốn vào hàm mất mát rồi hạ gradient trên chính đối tượng cần tạo ra — ở đó đối tượng là một cảnh 3D, ở đây là một ảnh. Ma trận Gram dùng chung nguyên lý hàm đối xứng với PointNet ở C/16-thi-giac-3d/02-bieu-dien-3d-va-depth. Mẫu đối kháng và trực quan hoá được nói kỹ ở E/25-do-tin-cay-va-van-hanh; mẫu hình dời chi phí sang lúc huấn luyện quay lại ở D/20-toi-uu-va-nen-mo-hinh [5] và ở phần chứng cất bộ lấy mẫu của C/17-mo-hinh-sinh/03-diffusion. Perceptual loss cũng chính là cầu nối sang C/17-mo-hinh-sinh/04-low-level-vision.

❏ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao cùng một đoạn code cho ra được cả một tác phẩm nghệ thuật lẫn một mẫu đối kháng — điều gì thực sự khác nhau giữa hai trường hợp?
2. Ma trận Gram bất biến với vị trí; hai thất bại nào của chuyển phong cách suy ra trực tiếp từ tính chất đó?
3. AdaIN chỉ khớp trung bình và độ lệch chuẩn theo kênh mà vẫn ra phong cách. Điều đó gợi ý gì về lượng thông tin thực sự cần thiết trong ma trận Gram?
4. Bạn thấy một ảnh trực quan hoá lớp rất đẹp trong một bài báo. Ba câu hỏi bạn phải hỏi trước khi tin nó là gì?

5. Mẫu hình “dời chi phí sang lúc huấn luyện” được mua bằng cái gì, và trong trường hợp cụ thể nào thì cái giá đó là không chấp nhận được?

Nguồn

- [1] L. A. Gatys, A. S. Ecker, and M. Bethge, “Image Style Transfer Using Convolutional Neural Networks,” in *CVPR*, 2016.
- [2] J. Johnson, A. Alahi, and L. Fei-Fei, “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” in *ECCV*, 2016.
- [3] X. Huang and S. Belongie, “Arbitrary Style Transfer in Real-Time with Adaptive Instance Normalization,” in *ICCV*, 2017.
- [4] J. Adebayo, J. Gilmer, M. Muelly, et al., “Sanity Checks for Saliency Maps,” in *NeurIPS*, 2018.
- [5] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NeurIPS Deep Learning Workshop*, 2015.